

版本说明

文档信息	文档名		RISC-V32 CPU综合设计	
	版本号		1.0	
	创建人		计算机组成与结构教学组	
更新记录				
序号	更新日期	更新人	版本号	更新内容
1	2026/06/25	牟正强	1.0	初版：适配RV32IM架构，增加特权指令、乘除法指令，引入riscv-tests测试集

文档错误或建议反馈:

本文档经过多版本迭代，但由于作者精力和能力有限，若发现错误或有改进建议请联系：

cstmzq@cqu.edu.cn

目 录

1 RISC-V CPU设计介绍	1
1.1 实验环境	1
1.2 实验目的	1
1.3 实验步骤	1
2 项目结构介绍	3
3 项目必做内容	4
3.1 添加RV32I剩余指令	4
3.1.1 算术逻辑运算指令	4
3.1.2 访存指令	5
3.1.3 分支指令	5
3.1.4 U型指令	5
3.2 实现动态分支预测器	6
3.2.1 两位饱和计数器	6
3.2.2 局部历史分支预测	7
3.2.3 全局历史分支预测	8
3.2.4 竞争分支预测	9
3.3 特权指令与异常处理	11
3.3.1 fence.i	11
3.3.2 CSR（控制状态寄存器）	12
3.3.3 异常处理	12
3.4 实现扩展指令	14
3.4.1 乘法指令	14
3.4.2 除法指令	15
3.4.3 取余指令	15
4 项目选做内容	17
5 项目评价标准	18
5.1 基础CPU设计	18
5.2 扩展内容	18
5.3 现场添加指令	18
5.4 现场答辩	19
5.5 设计报告	19
5.6 项目提交	20

6 测试说明	21
6.1 仿真测试	21
6.2 综合流程	22
6.3 上板流程	23
7 调试说明	24
7.1 仿真对比机制	24
7.2 调试小技巧	24
7.3 问题反馈	26

1 RISC-V CPU设计介绍

本次RISC CPU设计是建立在计算机组成与结构课程最后一个Cache项目的基础上进行，即假设你已经完成了如下设计：

1. 基础的RISC-V32架构的五级流水线CPU，支持lw、addi、slli、srai、sw、add、sub、and、or、slt、xor、beq等12条指令；
2. 处理器能较好处理数据冒险和控制冒险；
3. 实现了写回-写分配的数据Cache，在提供的基本SOC环境中能够跑通相应的测试用例。

本次设计就是在现有代码基础上进行“增量式”开发，增加RISC-V32I的其他指令（含特权指令）及乘除法扩展指令、增加分支预测功能、实现简单的异常处理、探索提升处理器性能的优化技术等。

1.1 实验环境

1. 开发软件：Vivado (版本需 ≥ 2025.1)；
2. 计算机平台：Windows或Linux 操作系统；内存尽可能达到8G以上；
3. 开发板：Nexys或Basys（可选）。

1.2 实验目的

1. 深化对RISC-V指令集架构的理解：掌握RISC-V 32I基本整数指令集（含特权指令）及乘除法扩展指令（M扩展）的编码格式、功能定义及其在流水线中的具体实现路径。
2. 掌握高级流水线优化技术：通过引入分支预测（如静态预测或动态分支预测）和前瞻执行机制，理解如何减少控制冒险带来的流水线停顿，提升处理器的指令级并行性。
3. 建立异常与中断处理的硬件思维：理解RISC-V特权架构中的异常与中断响应机制，掌握在流水线中实现精确异常处理（Precise Exception）的硬件设计方法（如刷新机制、CSR寄存器读写）。

1.3 实验步骤

1. 导入已完成五级流水线（12条指令）+ Data Cache的工程代码，确保项目完整且工程正确；
2. 添加RV32I剩余指令（详见第3章），进行仿真，能通过提供的测试用例；

3. 实现动态分支预测器，进行仿真，能通过提供的测试用例；对比增加分支预测器前后的处理器性能变化；

4. 添加异常与特权指令：实现必要的CSR（控制状态寄存器），增加特权指令的译码与执行，能够实现简单的异常处理功能。进行仿真，能通过提供的测试用例；

5. 实现扩展指令：增加乘法、除法和取余扩展指令，实现div/rem对除数为0的简单处理。进行仿真，能通过提供的测试用例；

6. 综合功能测试：进行仿真，能通过提供的测试用例；

7. 选做内容：学生根据自身基本情况，在扩展任务中（详见后文）至少选择一个内容进行实现。

（注意：2~6为必做内容，每一步的内容需建立在上一步的基础上才能完成。若小组未完成选做内容，则根据最终完成情况，最高成绩为良好）

2 项目结构介绍

我们提供的基础项目结构如表2.1所示，与Cache项目相似，红色部分需要重点关注。

表 2.1 项目结构

/docs	实验指导书、实验报告模板
/rtl	Verilog代码目录
/display	上板所需的数码管显示文件
/riscv_core	
/pipeline	流水线逻辑可以放在该目录下
/stages	可以存放IF、ID、EX、MEM阶段的核心模块，其中div和csr_file位于EX
bridge_1x2.v	转接桥
bridge_2x1.v	转接桥
cpu_axi_interface.v	将类SRAM信号转换为AXI信号
d_sram_to_sram_like.v	将CPU的数据请求信号转换为类SRAM信号
i_sram_to_sram_like.v	将CPU的指令请求信号转换为类SRAM信号
mmu.v	虚拟地址向物理地址转换
i_cache.v	指令Cache，已提供基础代码
d_cache.v	数据Cache，需要补全，可以沿用Cache项目的代码
/warp	
axi_wrap.v	AXI接口
axi_wrap_ram.v	访问RAM时的AXI信号
confreg.v	外设模块
/xilinx_ip	封装了时钟信号转换IP、AXI_RAM IP、PLL IP等
cpu_top.v	CPU顶层模块，连接RISC-V Core、mmu、i_cache和d_cache
soc_axi_lite_top.v	整个SOC顶层模块，控制仿真与综合逻辑
/run_vivado	
CQU-RISCVCPU-LAB2026.xpr	vivado工程运行文件
/syn	
soc_lite.xdc	综合时的约束文件
/sim	
mycpu_tb.v	仿真文件
/tests	测试用例

3 项目必做内容

这一章节给出每一实验步骤必须完成的内容，仅作一些重点提示，更多细节请同学们自行查阅相关资料。

3.1 添加RV32I剩余指令

本节内容参考《A01 RISC-V-Reader-Chinese-v2p12017.pdf》文档，每条指令的格式、功能可参考该书籍中“附录A RISC-V指令列表”。

3.1.1 算术逻辑运算指令

R型指令主要用于算术逻辑运算，我们已经实现了add、sub、and、or、slt、xor指令，只需在现有基础上增加其他指令即可。

0000000	rs2	rs1	000	rd	0110011	R add
0100000	rs2	rs1	000	rd	0110011	R sub
0000000	rs2	rs1	001	rd	0110011	R sll
0000000	rs2	rs1	010	rd	0110011	R slt
0000000	rs2	rs1	011	rd	0110011	Rsltu
0000000	rs2	rs1	100	rd	0110011	R xor
0000000	rs2	rs1	101	rd	0110011	R srl
0100000	rs2	rs1	101	rd	0110011	R sra
0000000	rs2	rs1	110	rd	0110011	R or
0000000	rs2	rs1	111	rd	0110011	R and

图 3.1 RV32I: R 型指令（10 条）

同理，对于相应的I型算术逻辑运算指令如图3.2所示。

imm[11:0]		rs1	000	rd	0010011	I addi
imm[11:0]		rs1	010	rd	0010011	I slti
imm[11:0]		rs1	011	rd	0010011	I sltiu
imm[11:0]		rs1	100	rd	0010011	I xori
imm[11:0]		rs1	110	rd	0010011	I ori
imm[11:0]		rs1	111	rd	0010011	I andi
0000000	shamt	rs1	001	rd	0010011	I slli
0000000	shamt	rs1	101	rd	0010011	I srli
0100000	shamt	rs1	101	rd	0010011	I srai

图 3.2 RV32I: I 型算术逻辑运算指令（9 条）

3.1.2 访存指令

RV32I访存指令分为取指令（I型，5条）和存指令（S型，3条），如图3.3所示。注意，我们的测试用例的数据是假设“大端对齐”的（主流处理器默认为小端序），所有在实现lh、lb等指令需要注意一下字节序：

比如对于数据“AABBCCDD”高位第一个字节应该是AA而不是DD.

imm[11:0]		rs1	000	rd	0000011	I lb
imm[11:0]		rs1	001	rd	0000011	I lh
imm[11:0]		rs1	010	rd	0000011	I lw
imm[11:0]		rs1	100	rd	0000011	I lbu
imm[11:0]		rs1	101	rd	0000011	I lhu
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	S sb
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	S sh
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	S sw

图 3.3 RV32I: 访存指令（8 条）

3.1.3 分支指令

分支指令包括无条件跳转指令（jal、jalr）和有条件跳转指令（B型，6条），如图3.4所示。无条件跳转是在ID阶段判断的，而B型指令可以选择在ID阶段，也可以在EX阶段。在我们的实验指导书中给出的ID阶段的版本，同学们也可以尝试更改为EX阶段实现。在现有数据通路中需要修改的即为立即数扩展模块和PC选择逻辑。

imm[20:10:1 11:19:12]				rd	1101111	J jal
imm[11:0]		rs1	000	rd	1100111	I jalr
imm[12:10:5]	rs2	rs1	000	imm[4:1:11]	1100011	B beq
imm[12:10:5]	rs2	rs1	001	imm[4:1:11]	1100011	B bne
imm[12:10:5]	rs2	rs1	100	imm[4:1:11]	1100011	B blt
imm[12:10:5]	rs2	rs1	101	imm[4:1:11]	1100011	B bge
imm[12:10:5]	rs2	rs1	110	imm[4:1:11]	1100011	B bltu
imm[12:10:5]	rs2	rs1	111	imm[4:1:11]	1100011	B bgeu

图 3.4 RV32I: 分支指令（8 条）

3.1.4 U型指令

imm[31:12]		rd	0110111	U lui
imm[31:12]		rd	0010111	U auipc

图 3.5 RV32I: U 型指令（2 条）

3.2 实现动态分支预测器

在计算机组成与结构的课程实验中，静态分支预测器采用“分支总不跳转”的固定策略，其硬件实现简单但预测准确率较低，特别是在循环或条件判断密集的程序中性能下降明显；作为对比，动态分支预测器则通过记录每条分支指令的历史执行行为（如采用一位或两位饱和计数器、分支历史表BHT或全局历史寄存器GHR）来动态调整预测方向，在运行时自适应地学习程序的分支模式，从而显著提高预测准确率，减少流水线因分支冒险带来的冲刷开销，是现代高性能处理器提升指令级并行的关键技术。

这里给出一个简单的示例，详细内容可参考docs文档中的《[A02 分支预测.pdf](#)》。

一、分支方向预测

3.2.1 两位饱和计数器

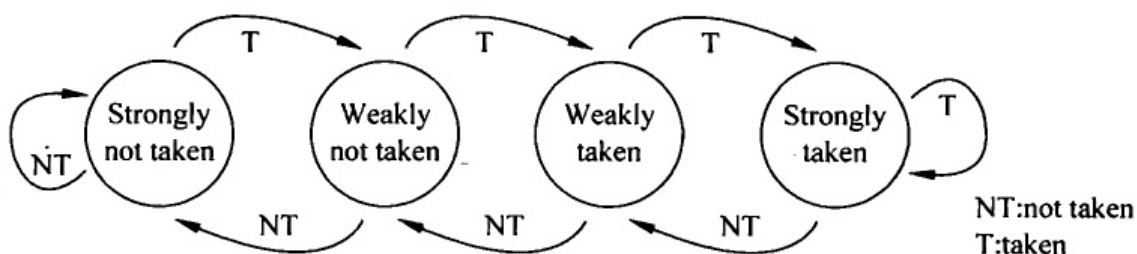
两位饱和计数器是最经典的分支方向预测器，具有4种状态：

SNT (00)：强不跳转，预测为 not taken

WNT (01)：弱不跳转，预测为 not taken

WT (10)：弱跳转，预测为 taken

ST (11)：强跳转，预测为 taken



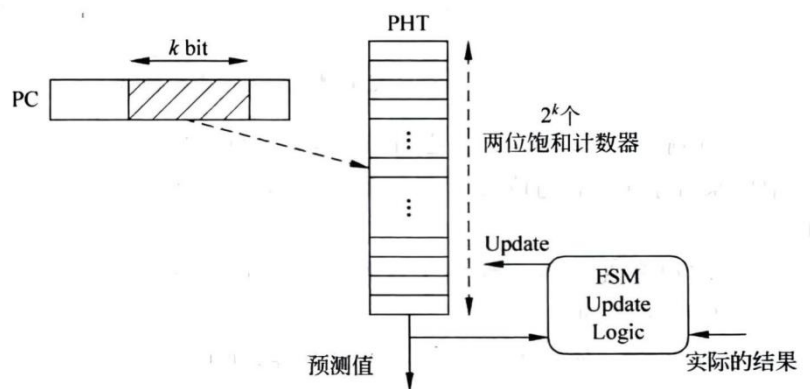
(1) 工作原理

- 使用PC的低k位作为索引，访问**模式历史表（PHT）**；
- PHT大小为 $2^k \times 2$ bit，每个表项是一个两位饱和计数器；
- 取指时，通过PC索引PHT，读取计数器状态并做出预测；
- 分支结果确定后（提交阶段），根据实际跳转情况更新计数器：
 - ✓ 分支实际跳转：计数器+1（向ST方向饱和）
 - ✓ 分支实际不跳转：计数器-1（向SNT方向饱和）

(2) 性能特征

PHT大小为2KB（即k=13）时，预测准确率可达93%以上；PHT大小通常设置为1Kb~4Kb（k取9~11位）；别名冲突会影响准确率，可通过Hash运算后再索引来缓解（

我们的处理器不涉及别名)。两位饱和计数器实现简单、硬件开销小，但无法捕捉分支指令的历史模式，对于循环等具有规律性的分支行为预测能力有限。



3.2.2 局部历史分支预测

(1) 核心思想

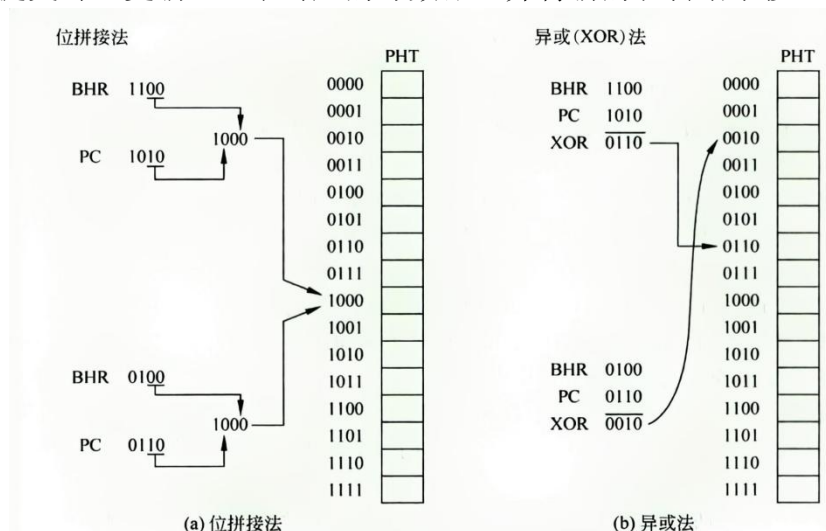
每条分支指令都有自身的执行历史模式，局部历史预测器利用这种模式来提高预测准确率。

(2) 硬件结构

- 分支历史表（BHT）：通过PC索引，每个表项是一个分支历史寄存器（BHR）；
- 每个BHR的位宽等于循环模式的周期长度，记录该分支最近N次的跳转历史（0表示not taken，1表示taken）；
- 用BHR的值作为索引，访问共享的PHT中的两位饱和计数器。

(3) 工作流程

- 取指时，用PC索引BHT，读出对应的BHR；
- 用BHR值索引PHT，读取两位计数器状态；
- 根据计数器状态做出跳转/不跳转预测；
- 分支提交时，更新PHT中对应的计数器，并将新的跳转结果移入BHR。



该方法的特点是：所有分支指令共享同一个PHT，减少硬件冲突。但只能依据自身指令的历史信息进行判断，无法利用其他分支的关联信息。对于周期性较强的循环分支效果较好，但预测能力有限，现已较少单独使用

3.2.3 全局历史分支预测

(1) 核心思想

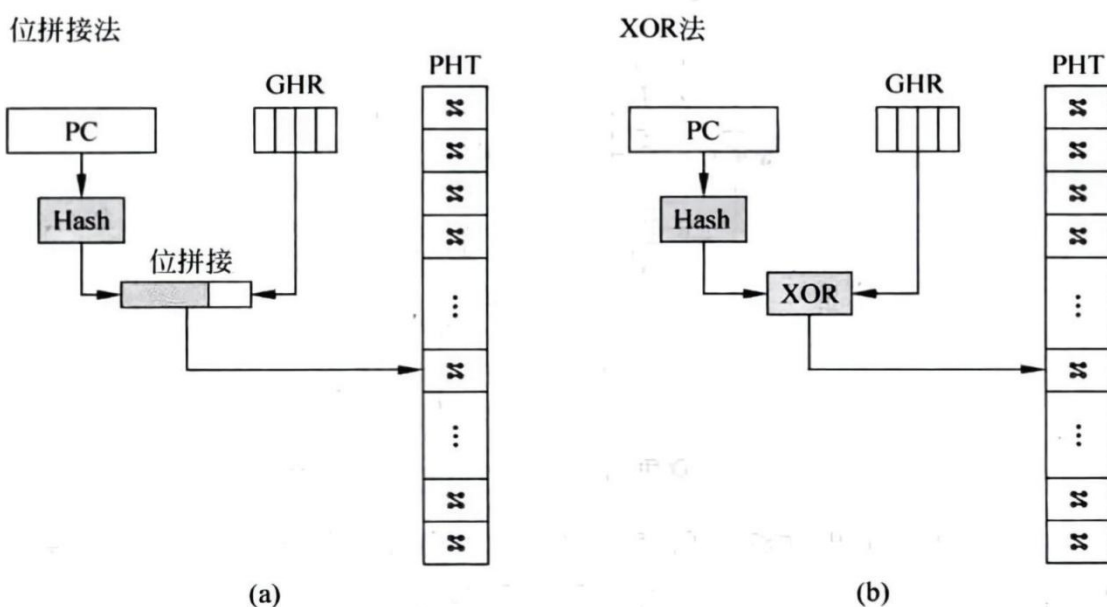
程序中的分支指令往往存在相关性——当前分支的跳转方向可能受前面若干条分支指令执行结果的影响。全局历史预测器正是利用这种跨分支的关联性来提高预测准确率。

(2) 硬件结构

- **全局历史寄存器（GHR）**：一个全局共享的移位寄存器，记录最近N条分支指令的执行结果（0表示not taken，1表示taken）；
- **模式历史表（PHT）**：用GHR的值作为索引进行寻址，每个表项是两位饱和计数器。

(3) 工作流程

- 每条分支指令执行时，其跳转结果移入GHR（新结果插入右侧）；
- 取指时，将GHR值作为索引访问PHT，读取计数器状态做出预测；
- 分支提交时，更新PHT中的计数器，并更新GHR。



为避免不同分支指令使用相同GHR值产生别名冲突，通常将PC与GHR进行Hash或XOR运算后再索引PHT。

全局历史预测器能够捕捉分支之间的关联性，对于高度相关的程序分支预测准确率较高，但无法利用单条分支指令自身的长期历史模式。

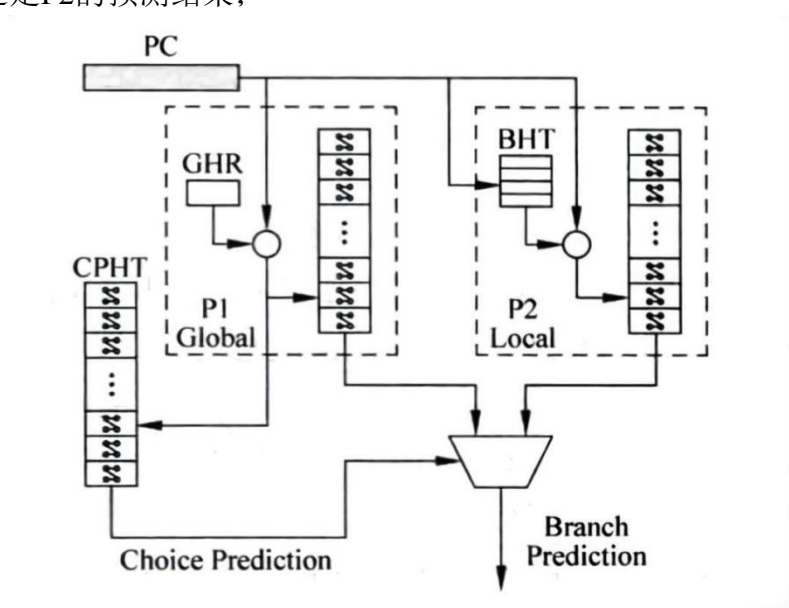
3.2.4 竞争分支预测

(1) 核心思想

局部历史预测器和全局历史预测器各有优势，适用于不同类型的程序。竞争预测器同时包含两种预测器，并通过一个选择器动态决定采用哪个预测结果。

(2) 硬件结构

- **全局历史预测器（P1）：** PC与GHR异或后索引PHT；
- **局部历史预测器（P2）：** PC索引BHT得到BHR，BHR再索引PHT；
- **选择预测器：** PC与索引异或后访问选择PHT（CPHT），由CPHT的状态决定采用P1还是P2的预测结果；



- **CPHT状态编码：** CPHT同样使用两位饱和计数器，状态为SNT/WNT选择P1（全局历史预测器）；状态为WT/ST选择P2（局部历史预测器）。

(3) 工作流程

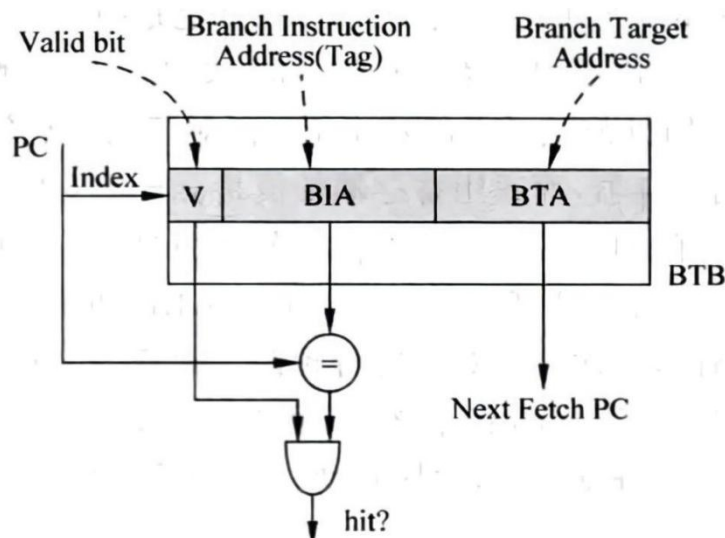
- 同时查询P1和P2，获得两个预测结果；
- 用PC索引CPHT，读取选择计数器的状态；
- 根据CPHT状态决定采用P1还是P2的预测；
- 分支提交时，根据实际跳转结果更新：
 - P1或P2中使用的预测器的计数器；
 - CPHT：如果P1正确而P2错误，向选择P1方向更新；反之向选择P2方向更新。

竞争预测器结合了全局和局部两种预测策略的优势，能够根据程序行为自适应地选择更优的预测方案，在现代高性能处理器中被广泛采用。

二、目标地址预测

方向预测解决了“跳不跳”的问题，但分支指令还需要预测“跳到哪里”的目标地址。

分支目标缓冲区（BTB）本质上是一个Cache，用于缓存分支指令的目标地址（BTA, Branch Target Address）。BTB使用PC的一部分作为索引，另一部分作为Tag进行匹配，存储存储分支指令的目标地址。组织形式可以使用直接映射，也可采用组相联设计。



(1) 工作原理

- 取指时，用PC索引BTB，同时比较Tag;
- 若命中：则读取存储的目标地址，预测PC为目标地址；对于直接跳转指令（ $PC + offset$ ），BTB中的目标地址可直接使用。
- 若缺失：流水线需要暂停，等待分支目标地址计算完成。对于直接跳转，等待时间较短；对于间接跳转（如函数指针），可能产生大量气泡。

(2) 设计优化

- 只存储跳转指令：只有实际发生跳转的分支指令才存入BTB，减少存储开销；
- Tag压缩：可采用异或方法降低Tag位数，提高存储效率；
- 目标地址预测：对于直接跳转，使用PC加偏移量计算，无需BTB也可快速得到目标地址。

(3) 硬件实现

BTB的实现方式和Cache类似，可以采用Reg寄存器，也可以利用BRAM资源。如果使用BRAM资源，那么就会有一个周期的延迟。建议最开始实现就采用寄存器方式。

三、基础要求

本次实验最低要求为实现两位饱和计数器预测器 + 直接跳转类型的BTB，基于局部历史和全局历史的预测器、间接跳转类型方法作为扩展内容之一。

3.3 特权指令与异常处理

RV32I涉及的特权指令如图3.6所示，以下所有指令均需实现（**fence指令可实现为空指令**）。在实现该部分内容时，请先仔细阅读《[403 RISC-V特权指令与异常处理介绍.pdf](#)》，可查阅相关资料加深对RISC-V特权架构的理解。

0000	pred	succ	00000	000	00000	0001111	I fence
0000	0000	0000	00000	001	00000	0001111	I fence.i
000000000000			00000	000	00000	1110011	I ecall
000000000001			00000	000	00000	1110011	I ebreak
csr			rs1	001	rd	1110011	I csrsw
csr			rs1	010	rd	1110011	I csrrs
csr			rs1	011	rd	1110011	I csrrc
csr			zimm	101	rd	1110011	I csrrwi
csr			zimm	110	rd	1110011	I csrrsi
csr			zimm	111	rd	1110011	I csrrci

mret

ExceptionReturn(Machine)

机器模式异常返回(Machine-mode Exception Return). R-type, RV32I and RV64I 特权架构从机器模式异常处理程序返回。将 *pc* 设置为 *CSRs[mepc]*，将特权级设置成 *CSRs[mstatus].MPP*，*CSRs[mstatus].MIE* 置成 *CSRs[mstatus].MPIE*，并且将 *CSRs[mstatus].MPIE* 为 1；并且，如果支持用户模式，则将 *CSR [mstatus].MPP* 设置为 0。

31	25 24	20 19	15 14	12 11	7 6	0
0011000	00010	00000	000	00000	1110011	

图 3.6 RV32I 特权指令

3.3.1 fence.i

*fence.i*指令格式如下，主要作用于我们的icache. 当执行*fence.i*时，处理器应使所有I-Cache行无效，强制后续取指时从主存中重新加载，以保证指令数据的一致性。

fence.i

Fence(Store, Fetch)

同步指令流(Fence Instruction Stream). I-type, RV32I and RV64I. 使对内存指令区域的读写，对后续取指令可见。

31	20 19	15 14	12 11	7 6	0
000000000000	00000	001	00000	0001111	

3.3.2 CSR（控制状态寄存器）

CSR它们不是普通寄存器 $x0 \sim x31$ ，而是 CPU 内部控制状态，详细介绍可参考 A03文档第5页内容。为了降低实现难度，我们提供了基本的CSR实现文件（csr_file.v），该模块需要添加到五级流水线的EX执行阶段，各个端口的含义如下：

```
module csr_file(
    // ===== 时钟与复位 =====
    input wire clk,                // 时钟信号
    input wire rst,                // 复位信号（高有效，异步复位）
    // ===== CSR 读写接口（来自译码/执行阶段） =====
    input wire [11:0] csr_addr_i,  // CSR 地址（12位，符合RISC-V CSR地址编码）
    input wire [31:0] csr_wdata_i, // CSR 写数据
    input wire csr_we_i,           // CSR 写使能（由CSR指令产生）
    input wire csr_write_intent_i, // 写操作意图标志（用于检查只读属性）
    output reg [31:0] csr_rdata_o,  // CSR 读数据
    output wire csr_illegal_o,      // 非法CSR访问标志：
    //   - CSR地址不存在
    //   - 当前特权级不足
    //   - 对只读CSR进行写操作
    // ===== 陷阱/异常入口接口（来自流水线） =====
    input wire trap_valid_i,        // 陷阱触发有效信号（异常/中断发生）
    input wire [31:0] trap_pc_i,    // 陷阱发生时的PC值（异常返回地址）
    input wire [31:0] trap_tval_i,  // 陷阱附加信息（如：非法指令编码、内存地址等）
    input wire [3:0] trap_cause_i,   // 陷阱原因编码（0-15，对应RISC-V异常码）
    // ===== 陷阱返回接口（来自流水线） =====
    input wire mret_valid_i,        // MRET指令有效信号（机器模式异常返回）
    input wire retire_i,            // 指令退休标志（用于更新性能计数器）
    // ===== 输出到流水线/异常处理单元 =====
    output wire [31:0] mtvec_o,     // 机器模式陷阱向量基址（低2位强制清零）
    output wire [31:0] mepc_o,      // 机器模式异常PC（当前异常返回地址）
    output wire [1:0] priv_mode_o   // 当前特权级输出（00=U-mode, 11=M-mode）
);
```

图 3.7 控制状态寄存器模块端口说明

图3.6中涉及的csrrw~csrrci 六条指令的读写操作均与该模块相关，对应的译码器和数据通路自行添加。

注意，提供的csr_file代码实现仅供参考，该模块的端口可以依据具体实现进行调整，但模块内部已有的控制状态寄存器不能进行随意修改，否则会影响仿真结果。

3.3.3 异常处理

本次设计主要处理异常，不处理外部中断，涉及到的异常类型说明如表3.1所示。其中异常1、3、5、7可以恒置为0或不实现。

表 3.1 RV32 异常类型说明（部分）

mcause	英文名称	中文名称	说明
0	Instruction address misaligned	指令地址未对齐	取指地址未按2字节或4字节对齐
1	Instruction access fault	指令访问错误	取指时发生物理保护或权限错误（如PMP禁止访问）
2	Illegal instruction	非法指令	译码时遇到无效指令编码或未实现扩展指令
3	Breakpoint	断点异常	执行ebreak指令触发，用于调试器断点
4	Load address misaligned	读地址未对齐	Load 指令的访存地址未按数据宽度对齐
5	Load access fault	读访问错误	Load指令触发PMP保护或物理存储访问错误
6	Store address misaligned	写地址未对齐	Store 指令的访存地址未按数据宽度对齐
7	Store access fault	写访问错误	Store指令触发PMP保护或物理存储访问错误
8	Environment call from U-mode	用户模式环境调用	U-mode 执行ecall指令，请求进入 M-mode 服务
11	Environment call from M-mode	机器模式环境调用	M-mode 执行ecall

在五级流水线中，异常在不同阶段被检测，但统一在提交阶段处理。当多个异常同时存在时，异常的处理顺序遵循流水线阶段先后顺序（IF > ID > EX > MEM），同阶段的异常则按RISC-V规范定义的优先级处理。

以上异常类型优先级从高到低为：0 > 1 > 2 > 3 > 8=11 > 4=6 > 5=7. 需要注意的是，指令地址未对齐异常包括IF阶段的取指或分支指令的目标地址（ID阶段或EX阶段），在实现时，为简化逻辑，可以只实现IF阶段的指令地址未对齐异常。各种异常的检测阶段通常为：

IF: 0、1;

ID: 2、3、8、11;

EX: 4、6

MEM: 5、7.

异常在对应流水级检测，沿流水传递；提交时（MEM/WB附近提交）只处理最老的一条异常，写入CSR（mepc/mcause/mtval/mstatus），PC跳转到mtvec，并Flush所有更年轻的流水线指令，保证精确异常。注意，不同异常写入CSR的值有所差异，请自行查阅相关资料实现不同的异常。

3.4 实现扩展指令

3.4.1 乘法指令

乘法指令属于RV32M扩展指令，本次设计要求实现mul、mulh、mulhsu、mulhu四条乘法指令（可以直接使用verilog内置的*运算），每条指令的格式如图3.8所示。

mul rd, rs1, rs2 $x[rd] = x[rs1] \times x[rs2]$

乘 (Multiply). R-type, RV32M and RV64M.

把寄存器 $x[rs2]$ 乘到寄存器 $x[rs1]$ 上，乘积写入 $x[rd]$ 。忽略算术溢出。

31	25 24	20 19	15 14	12 11	7 6	0
0000001	rs2	rs1	000	rd		0110011

mulh rd, rs1, rs2 $x[rd] = (x[rs1]_s \times_s x[rs2]) \gg_s XLEN$

高位乘 (Multiply High). R-type, RV32M and RV64M.

把寄存器 $x[rs2]$ 乘到寄存器 $x[rs1]$ 上，都视为 2 的补码，将乘积的高位写入 $x[rd]$ 。

31	25 24	20 19	15 14	12 11	7 6	0
0000001	rs2	rs1	001	rd		0110011

mulhsu rd, rs1, rs2 $x[rd] = (x[rs1]_s \times_u x[rs2]) \gg_s XLEN$

高位有符号-无符号乘 (Multiply High Signed-Unsigned). R-type, RV32M and RV64M.

把寄存器 $x[rs2]$ 乘到寄存器 $x[rs1]$ 上， $x[rs1]$ 为 2 的补码， $x[rs2]$ 为无符号数，将乘积的高位写入 $x[rd]$ 。

31	25 24	20 19	15 14	12 11	7 6	0
0000001	rs2	rs1	010	rd		0110011

mulhu rd, rs1, rs2 $x[rd] = (x[rs1]_u \times_u x[rs2]) \gg_u XLEN$

高位无符号乘 (Multiply High Unsigned). R-type, RV32M and RV64M.

把寄存器 $x[rs2]$ 乘到寄存器 $x[rs1]$ 上， $x[rs1]$ 、 $x[rs2]$ 均为无符号数，将乘积的高位写入 $x[rd]$ 。

31	25 24	20 19	15 14	12 11	7 6	0
0000001	rs2	rs1	011	rd		0110011

图 3.8 RV32M 乘法指令格式

3.4.2 除法指令

RV32M仅涉及div和divu两条除法指令，其指令格式如图3.9所示。

div rd, rs1, rs2 $x[rd] = x[rs1] \div_s x[rs2]$

除法(Divide). R-type, RV32M and RV64M.

用寄存器 $x[rs1]$ 的值除以寄存器 $x[rs2]$ 的值，向零舍入，将这些数视为二进制补码，把商写入 $x[rd]$ 。

31	25 24	20 19	15 14	12 11	7 6	0
0000001	rs2	rs1	100	rd	0110011	

divu rd, rs1, rs2 $x[rd] = x[rs1] \div_u x[rs2]$

无符号除法(Divide, Unsigned). R-type, RV32M and RV64M.

用寄存器 $x[rs1]$ 的值除以寄存器 $x[rs2]$ 的值，向零舍入，将这些数视为无符号数，把商写入 $x[rd]$ 。

31	25 24	20 19	15 14	12 11	7 6	0
0000001	rs2	rs1	101	rd	0110011	

图 3.9 RV32M 除法指令格式

需要注意的是，当除数为0时，**有符号除法**的结果直接返回0xFFFFFFFF(-1)；当除数为-1，被除数为0x80000000时，结果直接返回0x80000000。

3.4.3 取余指令

RV32M仅涉及rem和remu两条取余指令，其指令格式如图3.10所示。

rem rd, rs1, rs2 $x[rd] = x[rs1] \%_s x[rs2]$

求余数(Remainder). R-type, RV32M and RV64M.

$x[rs1]$ 除以 $x[rs2]$ ，向 0 舍入，都视为 2 的补码，余数写入 $x[rd]$ 。

31	25 24	20 19	15 14	12 11	7 6	0
0000001	rs2	rs1	110	rd	0110011	

remu rd, rs1, rs2 $x[rd] = x[rs1] \%_u x[rs2]$

求无符号数的余数(Remainder, Unsigned). R-type, RV32M and RV64M.

$x[rs1]$ 除以 $x[rs2]$ ，向 0 舍入，都视为无符号数，余数写入 $x[rd]$ 。

31	25 24	20 19	15 14	12 11	7 6	0
0000001	rs2	rs1	111	rd	0110011	

图 3.10 RV32M 取余指令格式

需要注意的是，当除数为0时，**有符号取余**的结果直接返回被除数；当除数为-1，被除数为0x80000000时，结果直接返回0。

verilog实现除法时不能直接像软件一样使用“/”或“%”运算符，因此我们提供了基于试商法的div.v除法运算器模块，执行完需要32个周期。同学们在实现时可以直

接使用该模块，优化触发器作为扩展内容之一。此外，因除法指令需要多个周期才能完成，所以在执行除法指令时必须暂停流水线。

4 项目选做内容

学生根据自身基本情况，在下列任务中**选择一至多个**内容进行实现。(如果没有完成选做内容的同学，根据最终完成情况，最高成绩不超过90分)

(1) 分支预测器进阶实现

我们最低要求仅要求实现基于两位饱和计数器、直接跳转类型的分支预测器。参考指导书，实现基于局部历史 或 全局历史 或 竞争分支预测 + 同时支持直接跳转类型和间接跳转类型的分支预测器。对比修改前后的性能差异。

(2) 除法器优化

课程设计给出**试商法**实现的除法运算器模块，执行完成需要32个周期。利用《计算机组成与结构》课程教授的**快速除法算法**，实现快速除法模块，替代现有除法器模块，优化除法指令消耗周期数。对比修改前后的性能差异。

(3) 硬件资源优化

目前icache、dcache均基于REG寄存器实现，会消耗大量的LUT资源。将这两个模块的buffer实现均修改为基于BRAM的方式实现（必须使用Block Memory Generator），对比两种方式的硬件资源消耗情况。

(4) Cache信号优化

取消当前项目中所有的类SRAM信号，全部修改为基于AXI总线的实现，并在icache中实现块八字 burst传输，对比优化前后的性能差异。**此项作为最为复杂扩展内容，实现即可获得优。**

(5) 关键路径优化

优化vivado综合工具报告的关键路径，尝试提升处理器主频（默认为50MHz）。各阶段组合逻辑部分完成功能复杂度有所不同，在译码、执行阶段的组合逻辑完成所需要的延时明显高于其他阶段，此时优化单一周期所需时间，尽可能使得功能复杂的阶段所实现的组合逻辑产生的延时降低，**优化单周期执行时间**。需提供优化前后的时序情况（WNS必须为正），FPGA资源消耗情况。

5 项目评价标准

成绩构成	任务描述		成绩占比
基础CPU设计	37条基础指令（不含特权与异常处理指令）	20分	50%
	动态分支预测器	10分	
	特权与异常处理指令	10分	
	扩展指令	10分	
扩展内容	至少一项	10分	10%
现场添加指令	现场随机抽取一条指令，实现并通过仿真	10分	10%
现场答辩	现场提问	10分	10%
设计报告	报告内容、代码重复率	20分	20%

5.1 基础CPU设计

- (1) 若37条基础指令未实现完整或未通过所有的测试用例，则最高成绩为合格；
- (2) 若实现动态分支预测器后无法通过所有测试用例，则按未通过比例扣0~10分；
- (3) 若添加特权和异常处理指令后无法通过对应的测试用例，则按完成情况扣0~10分；（注意，需要确保特权指令能正常运行才能测试扩展指令）
- (4) 若乘法指令未通过测试，扣5分；若除法、取余指令未通过测试，扣5分。

5.2 扩展内容

- (1) 若实现扩展内容无法通过功能仿真，得0分；
- (2) 若能通过仿真，但无法综合（如时序违例、严重错误、资源超限），得1分；
- (3) 若能通过仿真且能通过综合，时序通过且未存在critical warning的，得2~3分；
- (4) 若能通过仿真、综合、布局布线（Run Implementation）但WNS为负的，得4~5分；（允许降低CPU主频，但必须说明调整后的频率，调整方式参考测试说明章节）
- (5) 若能完成布局布线、生成bitstream、上板流程，频率低于45MHZ的，得6~9分。频率不低于45Mhz的，得10分。频率高于50MHz的，根据实际频率和FPGA数码管显示数值，额外提供0~10分的加分。

(该项目评价标准为试行标准，可能根据整体完成情况进行修改)

5.3 现场添加指令

每一组设计成员需要能够按照需要现场添加指令，其成绩评价标准如下表：

考核内容	评分标准
------	------

	90-100	80-90	60-80	60以下不合格
1. 添加指令 (80%)	在120分钟以内正确添加指令，并调试通过	添加的指令，但是功能未能完全正确运行，但是能够讲述设计思路，并分析其原因，代码基本正确	能够添加的指令，功能存在错误，原有数据通路设计存在一定缺陷造成难以添加指令。	不能现场添加指令，且无法说明具体原因的
2. 团队协作 (20%)	添加指令过程过程中分工明确，在各个工作环节配合良好	添加指令过程工作过程中分工较为明确，配合较好	添加指令过程有一定的分工，但是协作时不能发挥每位同学的作用	添加指令过程缺少团队组织，不能协同工作

5.4 现场答辩

小组随机抽取一人回答指令路径问题，具体要求如下：

考核内容 (权重)	评分标准			
	90-100	80-90	60-80	60以下不合格
1. 回答问题 (80%)	能够正确回答设计中所有问题，正确使用专业术语	能够回答所有关键性问题，但是存在理解上偏差，专业术语有少量差错	基本能够正确回答其中大多数问题，专业术语存在明显错误	不能回答设计文件中关键问题，不了解相关的专业术语
2. 团队协作 (20%)	回答问题时能够相互补充关键信息	能够按照熟悉的问题进行协作完成问答环节	答辩过程有一定的配合，但是缺少明显的分工协作	对于设计工作缺少明确的分工安排

5.5 设计报告

根据课程设计报告模板要求，课程设计报告得分采取减分制，缺少或不满足要求的，进行扣分，扣完为止。课程设计报告评分细则如下：

- (1) 报告模板中标注“必选”但未填写的，扣5分。
- (2) 模块设计中未写明信号名、输入输出类型、信号描述的，扣5分。

- (3) 全文中没有任何通路图的，扣10分；部分模块设计中缺少通路图的，扣5分。
- (4) 不填写工作日志的，扣10分，不按照模板要求填写的，扣5分。
- (5) 不填写主要错误记录，且CPU设计完成程度不足40分的，扣10分。
- (6) 参考文献格式不正确的，扣2分。

参考提供的参考书、文档、开源代码、其他同学的设计、AI辅助设计，但不在参考设计说明模块进行说明的，视作抄袭的，或班级内不同小组间代码高度雷同的，本次课程设计为不合格。

5.6 项目提交

1. 项目报告一份：

- 参考《课程设计报告模板.docx》撰写；
- 文件命名：“姓名1(学号1)_姓名2(学号2)_RISCVCPU设计”；
- 提交格式：保留word格式，无需转换为pdf；
- 每组只需提交一份即可。

2. 整个vivado工程项目（即CQU-RISCVCPU-LAB-2026）：

- **删除CQU-RISCVCPU-LAB-2026项目下的docs和tests文件夹**，保留如下文件夹，自己设计源码须放在rtl中对应的位置。

rtl	2026/6/28 21:02	文件夹
run_vivado	2026/6/28 21:07	文件夹
sim	2026/6/28 21:02	文件夹
syn	2026/6/28 20:48	文件夹

- **打开run_vivado文件夹，按照如下方式删除不必要的文件：**

.Xil	2026/4/30 0:17	文件夹	
cache_project_v0.1.cache	2026/4/30 21:00	文件夹	
cache_project_v0.1.hw	2026/4/30 21:00	文件夹	
cache_project_v0.1.ip_user_files	2026/4/30 21:00	文件夹	
cache_project_v0.1.runs	2026/4/30 21:00	文件夹	
cache_project_v0.1.sim	2026/4/30 21:00	文件夹	
cache_project_v0.1.srcs	2026/4/30 21:37	文件夹	
cache_project_v0.1	2026/4/30 21:41	Vivado Project F...	25 KB
dfx_runtime	2026/4/30 0:17	文本文档	1 KB
vivado.jou	2026/4/30 0:17	JOU 文件	4 KB

- 压缩包命名：“姓名1(学号1)_姓名2(学号2)_RISCVCPU设计”；
- 提交格式：将该工程文件夹压缩成zip（不包含报告）；
- 每组只需提交一份即可。